

AI Review and Verification

From Fast Code Generation to Trustworthy Software

Tang Liang

DigiPen Institute of Technology Singapore

AI Bootcamp, July 2026

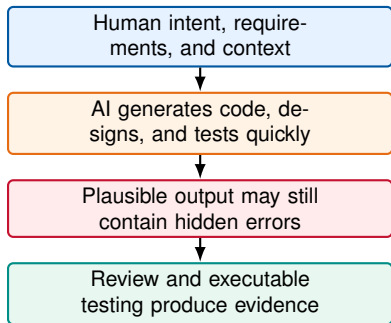
Why This Matters for 2026 Graduates

- Employers increasingly expect graduates to use AI tools productively.
- However, employability is not about “AI can write code for me”.
- The stronger signal is: **I can use AI and still produce correct, maintainable, verifiable software.**
- For RTIS/IMGD/CS graduates, this matters in games, simulation, tools, backend services, and real-time systems.

Professional positioning

AI-assisted engineers should be faster, but also more disciplined in review and verification.

The Core Problem: Code Is Easy to Generate, Hard to Trust



Main thesis

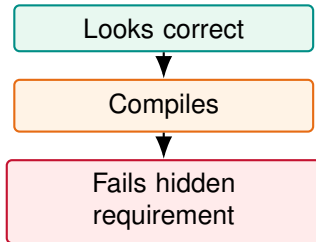
AI lowers the cost of generating code. Therefore, review and verification become the professional bottleneck.



AI can generate code faster than engineers can confidently review it.

Correct-Looking Code Can Still Be Wrong

- AI output often looks complete and confident.
- The code may compile but still violate requirements.
- The design may be reasonable but miss constraints.
- The tests may pass but cover only easy cases.



Engineering rule

Confidence is not evidence. Evidence comes from review, tests, logs, and reproducible verification.

A Quick Question: Is Confidence Enough?

AI says:

“The code is correct.”

But the output may only be confident, plausible, and incomplete.

Question: **Is this enough for professional software engineering?**

Confidence Is Not Evidence

AI says:

~~“The code is correct.”~~

Confidence is not evidence.

AI can produce confident answers very easily. Engineering requires proof, not confidence.

From AI Confidence to Engineering Evidence

What we should ask instead

- What requirement is satisfied?
- What tests passed?
- What edge cases were checked?
- What logs or CI results support this?
- Can another reviewer reproduce it?

Evidence

Review notes

Test cases

Regression result

Logs / CI output

Reproducible verification

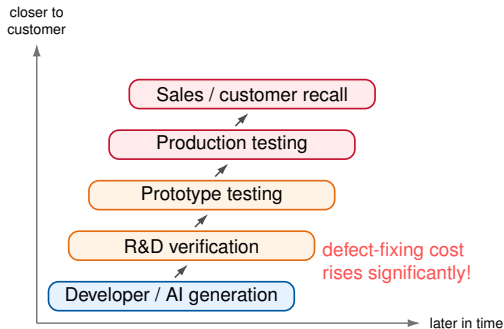
AI Review turns confident output into engineering evidence.

The Cost of Late Defect Discovery

- A common engineering heuristic: defect-fixing cost can grow by roughly an order of magnitude when defects escape to later lifecycle phases.
- Early review and testing are cheaper than field failure.
- AI-generated code increases the need for early defect detection.

Connection to AI review:

Fast code generation must be matched by early review, tests, logs, and reproducible verification.



From Coding Speed to Engineering Evidence

Weak AI story

- I used AI to write code.
- It looked correct.
- It ran once on my machine.

Strong AI engineering story

- I defined intent and constraints.
- I reviewed the AI output.
- I generated and reviewed tests.
- CI verified every change.
- I kept a traceable record of the prompt, design decision, test result, and final change.

What Is AI Review?

Definition for this lecture

AI Review is the systematic checking of AI-generated or AI-assisted engineering artifacts, including: requirements, prompts, design, code, tests, documentation, and CI/CD results.

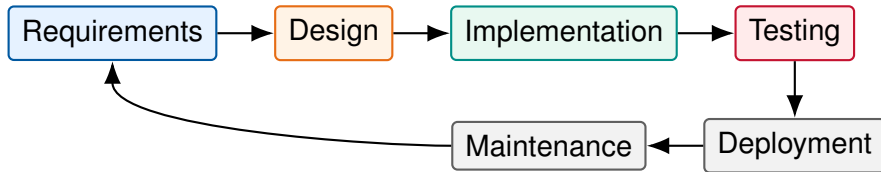
- It is not just proofreading AI output.
- It is not just asking another AI, “Is this correct?”
- It combines human judgement, AI critique, deterministic tools, and executable verification.

Lecture Roadmap

- 1 Background
- 2 SDLC in the AI Era
- 3 Software Verification Fundamentals
- 4 Reasoning-Centric Development
- 5 Practical AI Review Methods
- 6 CI/CD Concepts
- 7 AI + CI/CD
- 8 Governance Bridge and Takeaways

2. SDLC in the AI Era

Traditional Software Development Life Cycle



- The stages still describe real engineering concerns.
- AI changes the speed and granularity of iteration.

Why Traditional SDLC Existed

- Manual implementation was expensive.
- Teams needed staged planning and handover.
- Testing and release required specialised environments.
- Documents captured decisions for later maintenance.

Key point

The SDLC was not only a process diagram. It was a way to control engineering risk.

What AI Changes

Lower cost

- Code drafting
- Starter code and templates generation
- Test environment and test case setup
- Documentation drafts
- Build scripts

Higher pressure

- More code to review
- More generated assumptions
- More hidden defects
- Faster changes to integrate
- More need for regression tests

“The SDLC Is Dead?”

- Boris Tane’s article is useful as a provocative industry viewpoint.
- The argument: sequential phases are compressed by AI agents and fast feedback loops.
- The new loop emphasises intent, context, agent execution, observation, and iteration.

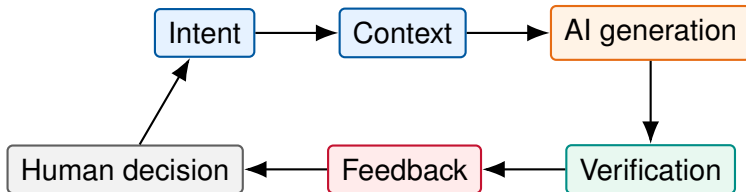
Interpretation

Manual sequential SDLC is weakened; continuous AI-assisted engineering loops are stronger.

Reference: Boris Tane, “The Software Development Lifecycle Is Dead”,

<https://boristane.com/blog/the-software-development-lifecycle-is-dead/>

AI-Assisted Lifecycle



Loop management

The engineer's task is to manage the AI-assisted development loop: intent, context, verification, and iteration.

- **Intent:** the goal, requirements, constraints, and expected behaviour.
- **Context:** the prompt, codebase, documents, test logs, previous decisions, and constraints given to the AI.
- **Verification:** the evidence that the generated result satisfies the intent.
- **Iteration:** the feedback loop that improves prompts, code, tests, and design decisions.

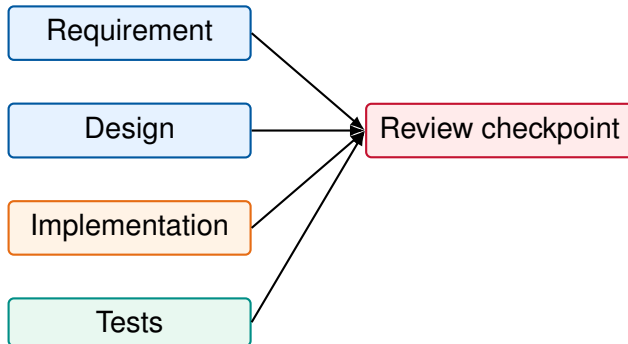
The Role of Context

- AI generation is highly sensitive to the provided context.
- Missing constraints produce wrong-but-plausible solutions.
- Good context includes requirements, interfaces, examples, constraints, previous decisions, and test expectations.

Practical message

A prompt is not just a question. In software engineering, it is a compact requirement and design context package.

Continuous Review



- Review should happen before the final large code diff.
- Review smaller artifacts when the cost of correction is still low.

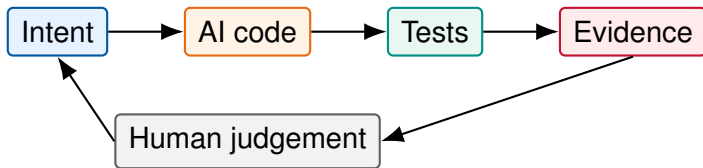
Continuous Verification

- Every generated change should be executable and testable as soon as possible.
- Regression tests protect previous functionality from AI-introduced changes.
- CI/CD creates repeatable, shareable verification evidence.

Key phrase

AI review becomes much stronger when it is connected to executable verification.

Key Takeaway: Manage the Loop, Not Just the Code



3. Software Verification Fundamentals

Verification vs Validation

Verification

Did we build it correctly?

- Matches specification
- Passes tests
- Meets interface contracts
- Meets performance constraints

Validation

Did we build the right thing?

- Solves user problem
- Fits real workflow
- Meets stakeholder needs
- Works in target environment

Testing Cannot Prove Everything

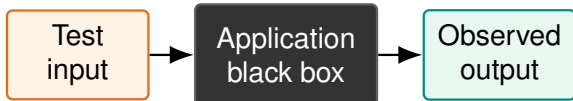
- Testing can reveal defects.
- Testing usually cannot prove the absence of all defects.
- Test quality depends on coverage, oracles, environment, and assumptions.

AI-specific risk

An AI can produce code that passes weak tests while still violating important requirements.

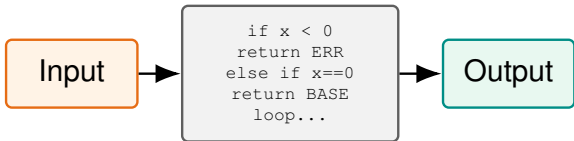
Black-Box Testing

- Treat the system as an opaque box.
- Focus on inputs, outputs, and requirements.
- Useful when internal implementation is unknown or not important.
- Strong for behaviour-level verification.



White-Box Testing

- Use knowledge of internal structure.
- Check branches, paths, internal states, and boundary logic.
- Useful for code coverage and targeted defect discovery.
- Important when AI-generated code has complex hidden assumptions.



Unit Testing

- Tests the smallest meaningful components: functions, classes, modules.
- Usually fast and repeatable.
- Good for algorithms, utility functions, and data transformations.

Example

For a pathfinding function: start equals goal, blocked path, shortest path, invalid coordinates.

Integration Testing

- Tests interaction between modules.
- Finds interface mismatch and data format problems.
- Often more important after AI generation because different files may be generated with inconsistent assumptions.

Typical failures

Wrong file path, wrong JSON format, wrong units, wrong coordinate convention, wrong error code.

System Testing

- Tests the full system as users or operators experience it.
- Includes UI, backend, algorithm, data files, and deployment environment.
- In your lab: browser + Flask + C++ executable + Python verification + Jenkins.

Regression Testing

- Confirms that new changes did not break previous behaviour.
- Essential for iterative AI-assisted development.
- Converts old bugs into permanent tests.

Professional habit

When a defect is found, add a regression test before or during the fix.

Edge Cases

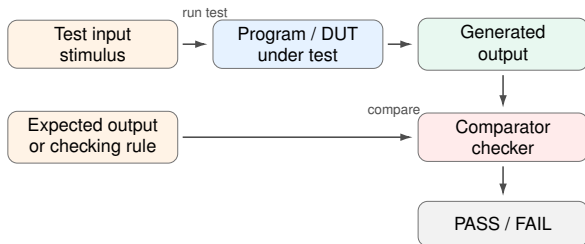
- Empty input
- Minimum and maximum values
- Invalid format
- Duplicates
- Negative values
- Timeout or large input

AI review prompt

Find boundary cases, invalid inputs, and counterexamples that may break this implementation.

How Do We Know Pass or Fail?

- A test needs a clear input stimulus.
- The program under test produces an actual output.
- We also need an expected output or checking rule.
- PASS/FAIL is decided by comparison.



Key idea

A test is meaningful only when we know what the correct result should be.

What Is a Test Oracle?

- In software testing, a **test oracle** is the expected result or checking rule used to decide PASS/FAIL.
- It plays a similar role to the **golden output** or **reference checker** in DUT verification.
- A test oracle may come from:
 - a formula;
 - a trusted reference implementation;
 - a specification;
 - pre-generated expected output;
 - carefully reviewed human judgement.
- Without a reliable oracle, a test may only show that the program runs, not that it is correct.

AI-specific risk

If AI generates both the implementation and the expected result, it may repeat the same misunderstanding in both places.

Code Coverage: Useful, But Not Proof

- Code coverage measures which parts of the code were executed by tests.
- Common question: did our test suite actually reach this function, line, branch, or state?
- High coverage does **not** prove that the design is correct.
- A test may execute code but still miss wrong requirements, weak assertions, or incorrect expected output.
- Low coverage is a warning: unexecuted code has not been tested by the current test suite.

Key message

Coverage tells us where tests have been. It does not tell us whether the design is correct.

Common Coverage Metrics

- **Line coverage:** did each source-code line execute?
- **Statement coverage:** did each executable statement execute?
- **Function coverage:** was each function or method called?
- **Branch coverage:** did each branch direction execute?
- **Condition coverage:** did each Boolean sub-condition evaluate true and false?
- **Path coverage:** were different execution paths exercised?
- **State / transition coverage:** were important states and transitions reached?
- **MC/DC:** did each condition independently affect the decision result?
- **Functional coverage:** were required scenarios or behaviours exercised?

Practical note

In normal software projects, line and branch coverage are common. In critical systems, stronger metrics may be required.

Coverage Requirements Vary by Domain

Critical hardware / software

- High coverage targets are often expected.
- Uncovered code may need justification or waiver.
- Late defects can be extremely expensive.
- Some products are difficult or impossible to update after release.
- Examples: ASIC, FPGA, firmware, telecom infrastructure, automotive, aerospace, medical systems.

General software products

- Coverage is useful but often less strictly enforced.
- Products can often be patched after release.
- Teams may prioritise feature delivery, integration tests, monitoring, or user feedback.
- Coverage targets vary by company, product risk, and engineering culture.

Key message

The stricter the cost of failure, the stronger the verification evidence usually needs to be.

Why Some Software Teams Resist Coverage

- Coverage can be gamed: tests may execute code without checking meaningful correctness.
- High line coverage does not guarantee edge-case, concurrency, security, or performance correctness.
- UI code, framework glue code, generated code, and logging wrappers can be expensive to cover well.
- Fixed coverage targets may create bad incentives, such as shallow tests written only to increase the percentage.
- Fast-moving product teams may prioritise delivery speed, end-to-end tests, monitoring, and customer feedback.
- Large software systems may fail because of data, network, timing, configuration, or integration issues that line coverage does not capture.

Balanced view

Do not treat coverage as proof. Do not ignore it either. Use coverage to find blind spots, then add meaningful tests and checks.

Coverage Analogy: Attendance

Student attendance

- If a student never attends, we cannot assess their in-class performance.
- If a student attends, it does not prove they are a good student.
- Attendance only shows that assessment was possible.

Code coverage

- If code is never executed, it is not tested by the test suite.
- If code is executed, it does not prove the code is correct.
- Coverage only shows that checking was possible.

Key message

Coverage is necessary evidence for test reach, but correctness still needs assertions, expected outputs, and review.

Coverage Tools

C / C++

- gcov / lcov
- llvm-cov
- OpenCppCoverage
- BullseyeCoverage
- SonarQube / SonarCloud

Python

- coverage.py
- pytest-cov

C# / Unity

- Unity Test Framework coverage support
- Visual Studio Code Coverage
- coverlet
- JetBrains dotCover

CI/CD reporting

- Jenkins coverage reports / plugins
- GitHub Actions coverage reports
- GitLab CI coverage reports
- Codecov / Coveralls
- SonarQube quality gates

Coverage in AI-Assisted Development

- AI can generate many tests quickly, but some tests may be shallow.
- Coverage reports help identify untested functions, lines, branches, and states.
- Reviewers should ask whether the tests check correct behaviour, not only whether they execute code.
- Coverage should be combined with test oracles, assertions, regression tests, code review, and CI/CD logs.

AI review question

Did AI improve real verification quality, or only increase the coverage number?

Static Checks and Static Review

- **Static checks** inspect code without executing the full program.
- Examples: compiler warnings, linters, type checkers, formatters, and security scanners.
- These tools are usually deterministic and repeatable.
- **AI-assisted static review** also inspects code without running it, but it is not deterministic.
- Static tools and AI review should complement each other, not replace dynamic testing.

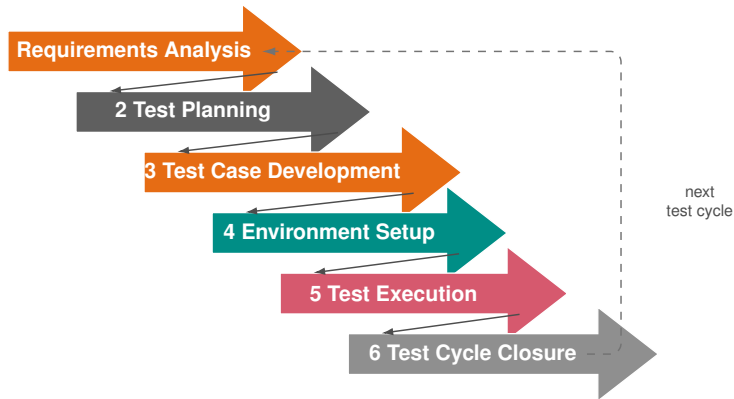
Tool examples

```
g++ -Wall -Wextra, clang-tidy, cppcheck, ruff, mypy
```

Key message

AI can help identify suspicious logic, missing cases, and design risks, but compiler/linter results and executable tests remain necessary evidence.

Software Testing Life Cycle



Verification Evidence

- Build logs
- Test reports
- Screenshots or output files
- Coverage reports
- Code review comments
- Prompt/context logs for AI-generated changes
- CI results: PASS / FAIL

Mindset

Evidence beats confidence.

4. Reasoning-Centric Development

The Role of Engineers Is Shifting

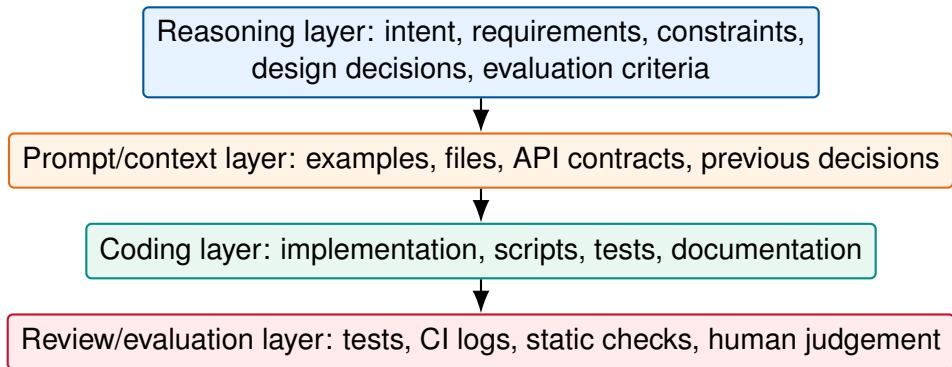
Old emphasis

- Write most code manually
- Debug syntax and implementation
- Submit code diff
- Reviewer checks implementation

AI-assisted emphasis

- Define intent and context clearly
- Guide AI-generated implementation
- Review reasoning, code, and tests
- Preserve traceable evidence

Coding Layer vs Reasoning Layer



The New Skill Stack

- 1 Problem framing
- 2 Requirement clarification
- 3 Prompt and context preparation
- 4 AI generation
- 5 Code/design review
- 6 Test design
- 7 Regression verification
- 8 Debugging and iteration
- 9 Documentation of reasoning

Interview-ready statement

I used AI to accelerate implementation, but I controlled the specification, review, tests, and verification evidence.

Git Is Necessary, But Not Enough

- Git records snapshots and diffs of files.
- In AI-assisted development, the final diff may not explain why the code was generated.
- Important context may disappear when the AI session ends.
- Therefore, prompts, design notes, review notes, and test plans should be preserved.

Practical rule

If AI-generated code is committed, the prompt/context and verification evidence should also be versioned.

Old Workflow to New Workflow

Traditional workflow	AI-assisted workflow
Humans write code	Agents draft code; humans define intent and review
Humans review code diff	Humans review intent, reasoning, diff, and evidence
Git stores code history	Need code history plus prompt/session history
Pull request is collaboration unit	Agent session may also become a collaboration unit
Knowledge is in developer's mind	Knowledge must be externalised into context/docs
Review asks "what changed?"	Review also asks "why was it changed?"

What Is a Pull Request?

- A Pull Request, or PR, is a request to merge changes from one branch into another.
- It is an important review milestone in a software project.
- A PR usually includes the code diff, discussion, reviewer comments, approvals, and CI results.
- In traditional development, the PR is often the main collaboration unit for code review.

Connection to AI-assisted development

With AI agents, we may also need to review the prompt, context, reasoning record, and verification evidence behind the code diff.

What Is an Agent Session?

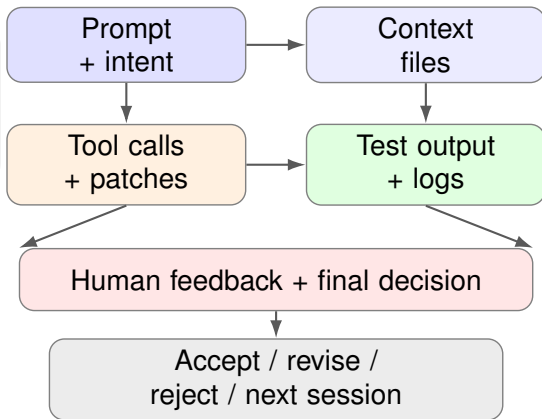
Definition

An **agent session** is one complete AI-assisted working episode: from prompt and context, through agent actions and tool use, to test results and human decision.

- Prompt and intent
- Context files
- Tool calls
- Files touched
- Generated patches
- Test output and logs
- Human feedback

Key idea

An agent session preserves the decision and evidence trail behind the final code.



Code diff tells us **what changed**;
agent session records **what was requested,**
performed, verified, and approved.

Industry Direction: Checkpoints

- AI-assisted development may involve many prompts, files, tool calls, and test results.
- If this information disappears, later review and debugging become difficult.
- A checkpoint preserves the important engineering record at a meaningful point in the work.
- Useful checkpoints may include:
 - prompt and intent;
 - relevant context files;
 - design decision;
 - code change;
 - test result;
 - human review decision.

Practical takeaway

For important AI-assisted changes, keep a short Markdown record of the prompt, decision, test result, and review notes.

Why Checkpoints Matter

Without checkpoints	With checkpoints
AI session context disappears	Prompt/session context is saved
Agent may repeat the same mistake	Previous decisions can be reviewed
New developer starts from zero	Handover becomes smoother
Reviewer only sees final diff	Reviewer can inspect intent and path
Debugging is code-centric	Debugging becomes reasoning-centric
Token cost is wasted repeating context	Past context can be reused

Commit Prompt and Markdown Files

- Requirements
- Prompt log
- Design decisions
- Test plan
- Review notes
- CI result summary

```
project/  
  cpp/  
  python/  
  app.py  
  docker-compose.yml  
  docs/  
    requirements.md  
    ai_prompt_log.md  
    design_decisions.md  
    test_plan.md  
    review_notes.md
```

Preserve Evidence Without Exposing Sensitive Data

- Do not commit passwords, API keys, access tokens, or other credentials.
- Do not send proprietary code, personal data, or complete production logs to an unapproved AI service.
- Treat retrieved documents, webpages, issue text, and logs as **untrusted input**; they may contain malicious or misleading instructions.
- Remove or hide sensitive information before saving prompts, logs, screenshots, or AI-session records.
- When the raw conversation is sensitive, preserve an approved summary of the intent, relevant context, decisions, and verification evidence.
- Record the AI tool or model and date because its behaviour may change.

Practical rule

Preserve enough evidence for review and reproduction—but never preserve secrets or restricted data merely for completeness.

Prompt Log Template

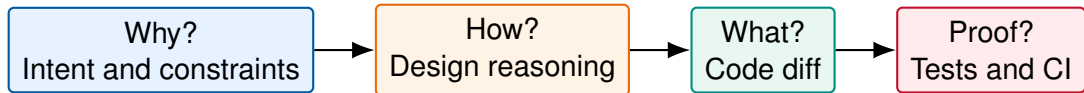
```
# AI Prompt Log

## Session 1: Initial Design
Goal:
Context given to AI:
Prompt used:
AI output summary:
Human review:
Verification result:
Known issues:
Next action:
```

Decision and evidence trail

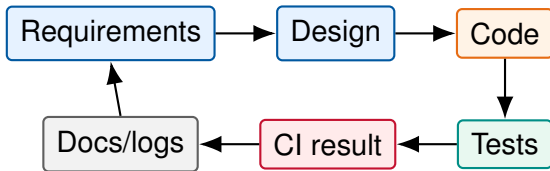
It is more useful to record why the change was requested, how it was reviewed, and what evidence supports it than to inspect only the final code.

Takeaway: Review the Why, Not Only the What



5. Practical AI Review Methods

AI Review Method Map



- AI review can target every AI-assisted work product, not only source code.

Method 1: Cross-AI Review

- Use one AI tool to generate and another tool to critique.
- Ask a third tool to propose tests, counterexamples, and missing assumptions.
- Compare disagreements and resolve them using engineering judgement.

Example workflow

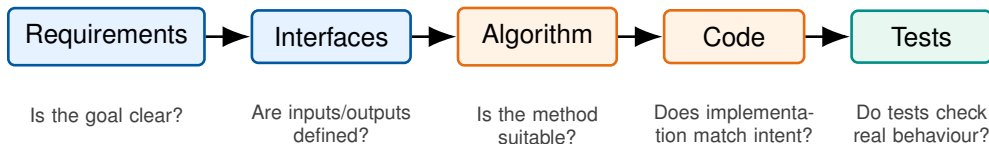
Claude Code writes the implementation; ChatGPT reviews it; Gemini suggests tests; Jenkins executes the verification.

Important limitation

A second AI provides hypotheses, counterexamples, and review questions—not independent proof. Different models may reproduce the same popular but incorrect assumption. Verify their claims using trusted test oracles and executable evidence.

Method 2: Checkpoint-Based Review

- Review at meaningful checkpoints instead of waiting for one large final review.
- Each checkpoint should have a clear review question and evidence.
- This reduces the risk of accepting a wrong AI-generated direction too late.



Key message

Checkpoint-based review catches misunderstandings early, before they become expensive implementation or testing problems.

Method 3: Specification-Based Review

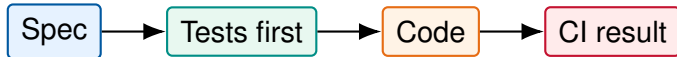
- Give AI the explicit specification and the implementation.
- Ask it to identify mismatches, missing cases, and ambiguous assumptions.

Prompt pattern

Review this implementation against the following specification. List each requirement, whether the code satisfies it, and evidence or counterexample.

Method 4: Test-First AI Development

- Ask AI to generate test cases before implementation.
- Human reviews the test cases.
- AI generates code to satisfy the reviewed tests.



Method 5: Adversarial Review

- Do not ask AI to praise the solution.
- Ask AI to attack the design and find failure modes.

Prompt pattern

Act as a strict reviewer. Do not describe strengths. Find defects, missing assumptions, unsafe cases, and tests that would break this solution.

Method 6: Counterexample Search

- A **counterexample** is a valid case that breaks a claim, assumption, or implementation.
- It is often an edge case, boundary case, or unusual but allowed input.
- Example claim: “the network send operation always sends useful payload data.”
- Counterexample: a WinSock send call with data length equal to zero.
- The goal is to find cases where the program’s logic no longer matches the intended behaviour.

Key message

Do not only test normal cases. Ask AI to find valid cases that challenge your assumptions.

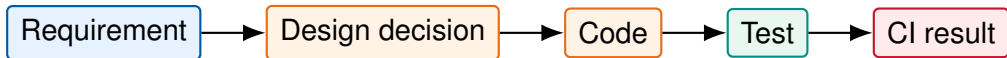
Method 7: Review the Tests

- AI-generated tests may be shallow.
- Tests may encode the same mistaken assumption as the implementation.
- Review test names, expected results, edge cases, and test oracle.

Question

Do the tests prove the requirement, or only prove that the current code behaves consistently with itself?

Method 8: Traceability Review



- Every important requirement should connect to implementation and test evidence.

Method 9: Static-Tool-Supported AI Review

- Use deterministic tools first; ask AI to explain and prioritise issues.
- Static tools are good at consistency, style, type, and simple defect classes.

Example

Run `g++ -Wall -Wextra`; paste warnings to AI; ask for root cause and minimal safe fix.

Method 10: Regression Review

- Every AI-generated change should pass existing tests.
- If a bug is found, add a test to prevent recurrence.
- Regression tests convert learning into executable memory.

AI-era problem

Without regression tests, an agent may fix one bug while reintroducing an old one.

Method 11: Interface Review

- Check interface agreements between components: inputs, outputs, formats, errors, timing, and ownership.
- Especially important when AI edits multiple files.

Check

- Input format
- Output format
- Error handling
- Units and coordinate systems

Failure examples

- JSON field mismatch
- Row/column swapped
- Zero-based vs one-based indexing
- Seconds vs milliseconds

Method 12: Performance Review

- AI may generate correct but inefficient code.
- Review time complexity, memory use, scalability, and real-time constraints.
- For RTIS/game/simulation students, performance is part of correctness.

Prompt pattern

Analyse the time and memory complexity. Identify cases where this implementation may become too slow or unstable.

Method 13: Maintainability Review

- AI code can be verbose, inconsistent, or over-engineered.
- Review naming, modularity, comments, error handling, and unnecessary abstraction.
- Maintainability affects future debugging and team collaboration.

Method 14: Human Approval Gates

- AI can suggest, but humans remain accountable.
- Use approval gates for architecture, external APIs, data handling, performance-critical code, and deployment.

Boundary

Require human approval for consequential or difficult-to-reverse actions. For example, do not let an agent autonomously change important architecture or deployment settings without human review.

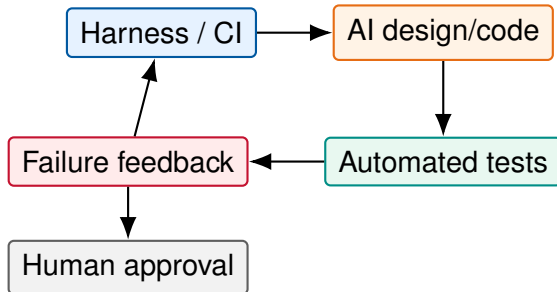
Before an AI Agent Acts

- **Scope the task:** define the objective, constraints, permitted files, and expected output.
- **Limit access:** allow only the tools, data, systems, and permissions required for the task.
- **Use isolation:** run untrusted or generated code in a sandbox or controlled environment.
- **Require approval:** pause before deployment, deletion, payment, external communication, or access to sensitive data.
- **Record actions:** retain relevant tool calls, file changes, test results, and human decisions.
- **Verify independently:** inspect the final state using trusted tests, deterministic tools, and human review.

Agent-control principle

The more autonomy and system access an agent receives, the stronger its boundaries, monitoring, approval gates, and verification must be.

Method 15: Agentic Feedback Loop



- Advanced form: test result drives the next AI improvement.
- Keep human control for important decisions.

Reusable AI Review Prompts

Why this matters

AI review methods become useful only when we can ask clear review questions. The prompts below are reusable patterns for asking AI to review software artifacts.

1. Review this code against the specification below.
2. Find counterexamples that break this algorithm.
3. Generate edge-case tests and explain the expected outputs.
4. Check interface consistency across these files.
5. Analyse the CI failure log and propose the minimal fix.
6. Identify what assumptions are not stated in the requirements.

AI Review Checklist

- What changed?
- Why was it changed?
- What requirement does it satisfy?
- What assumptions did AI make?
- What tests cover it?
- What CI evidence is available?
- What human decision was made?

6. CI/CD Concepts

Why CI/CD Matters for AI Review

- AI review can produce useful critique, but CI/CD produces repeatable execution evidence.
- CI/CD makes testing systematic, visible, and shareable.
- It prevents “works on my machine” from becoming the final standard.

Key link

AI review tells us what might be wrong. CI/CD tests whether the system still works.

Continuous Integration

- Integrate changes frequently.
- Build and test every meaningful change.
- Detect integration errors early.



Continuous Delivery / Deployment

- **Continuous Delivery:** software is always kept in a releasable state after passing automated checks.
- **Continuous Deployment:** passing changes may be automatically released to users or production systems.
- In critical systems, deployment is usually controlled by release windows, approval gates, rollback plans, and monitoring.
- Some online systems require near-zero downtime; others allow only a short maintenance window.
- Example: telecom infrastructure upgrades may affect city-scale service, so deployment must be carefully scheduled, tested, and recoverable.

Key message

CI/CD is not only automation. It is controlled, repeatable, and evidence-based software delivery.

Pipeline Stages



- Pipelines convert engineering expectations into repeatable checks.

Jenkins

- Jenkins is an automation server.
- It can run build scripts, test scripts, and report success/failure.
- In this bootcamp, Jenkins is not installed as a standalone application.
- It runs inside Docker as part of the lab environment.

Docker

- Docker packages software and dependencies into containers.
- It reduces environment mismatch across machines.
- It is useful for reproducible teaching labs.

Lab connection

Docker gives us a reproducible environment; Jenkins gives us repeatable verification.

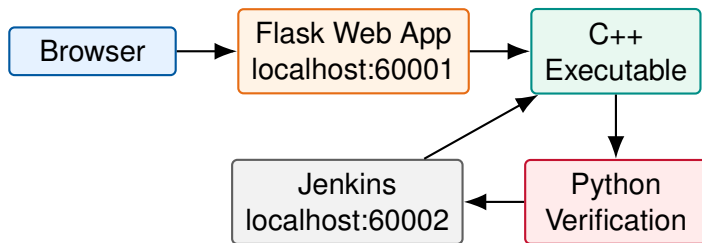
Docker Compose

- Docker Compose starts multiple containers using one configuration file.
- Useful when the lab needs a web app, Jenkins, and shared workspace.
- Users can run the environment with a small number of commands.

Typical command

```
docker compose up -build -d
```

Lab Demo Architecture



Jenkins Demo: SUCCESS

- Correct implementation: `return a * b;`
- Jenkins compiles the C++ program.
- Python verification script checks the expected outputs.
- Build result: SUCCESS.

Key message

A successful CI result is verification evidence, not just a green icon.

Jenkins Demo: FAILURE

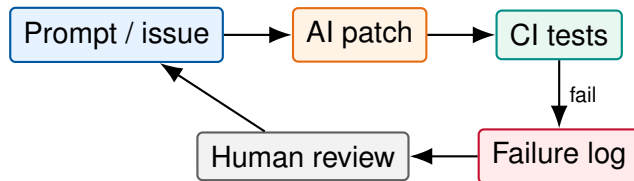
- Inject bug: change $a * b ==> a + b$.
- Jenkins rebuilds and reruns verification.
- Test fails and Console Output shows diagnostic information.

AI review connection

The failure log can be given to AI to diagnose root cause and propose a fix.

7. AI + CI/CD

AI-Assisted CI/CD Loop



- The loop is useful only if tests are meaningful.

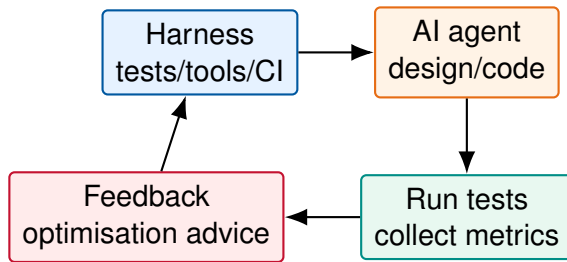
From Test Result to AI Diagnosis

- Paste the failing command, log, expected behaviour, and relevant code to AI.
- Ask for root cause, not only a patch.
- Ask for minimal fix and additional regression test.

Prompt pattern

Given this CI failure log and the intended behaviour, identify the root cause, propose the minimal fix, and suggest a regression test.

Agentic Flywheel



Meaning

A closed loop where automated feedback guides the next AI-generated improvement.

Why the Harness Matters

- The harness defines what the agent can observe and optimise.
- Good tests create useful feedback.
- Weak tests create false confidence.
- The agentic flywheel is only as good as the verification harness.

Weak Tests Create False Confidence

Weak test suite

- Only happy path
- No invalid input
- No edge cases
- No regression cases

Strong test suite

- Normal cases
- Boundary cases
- Failure cases
- Regression cases
- Performance cases when needed

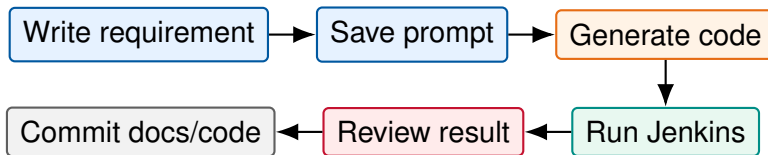
Human-in-the-Loop Control

- AI can generate fixes quickly.
- CI can run checks automatically.
- Humans still decide whether the fix is appropriate, maintainable, and aligned with intent.

Engineering principle

Do not confuse automation with accountability.

Practical Workflow



Employability Message

Do not only say

I used ChatGPT or Copilot to write code.

Say instead

I used AI to accelerate implementation, preserved the prompt and design rationale, reviewed the output, created regression tests, and verified the result through CI.

8. Governance Bridge and Takeaways

Governance: Brief Bridge Only

- Governance includes security, privacy, IP, licensing, compliance, and organisational policy.
- Another lecture covers this in detail.
- For this lecture, remember: verification is technical evidence; governance defines acceptable use and risk boundaries.

Final Checklist

- Intent clearly defined?
- Context and constraints provided?
- Prompt/session history saved?
- AI output reviewed?
- Tests reviewed?
- CI/CD result available?
- Regression risk checked?
- Human decision recorded?

Final Takeaway

Be AI-accelerated

Be verification-grounded

Be reasoning-centric

The future software engineer is not only a code writer.
The future software engineer is a builder of
trustworthy systems.

Suggested Reading

- Boris Tane, “The Software Development Lifecycle Is Dead”, <https://boristane.com/blog/the-software-development-lifecycle-is-dead/>
- Entire, “Hello Entire World”, <https://entire.io/blog/hello-entire-world>
- Entire homepage, <https://entire.io/>
- DORA research program and reports, <https://dora.dev/>